

STATEMENT OF WORK

PRODUCT 004 — FRESHERS

Restaurant OpenAPI + SQLite Interface Harness

Learn the Contract. Implement the Backend. Let the Harness Prove It.

6 HOURS MAXIMUM — DELIVER OR NOTHING

1. FINISHED PRODUCT

Build a simple reusable development Harness for OpenAPI-first HTTP interfaces backed by SQLite, then prove the Harness by implementing a canonical Restaurant backend whose behavior matches the supplied OpenAPI specification.

```
openapi.yaml = CONTRACT / SOURCE OF TRUTH
  ↓
Fresher implements handlers + SQLite access
  ↓
Harness validates spec + resets DB + runs API + tests request/response contracts
  ↓
PASS = implementation matches OpenAPI + business rules
```

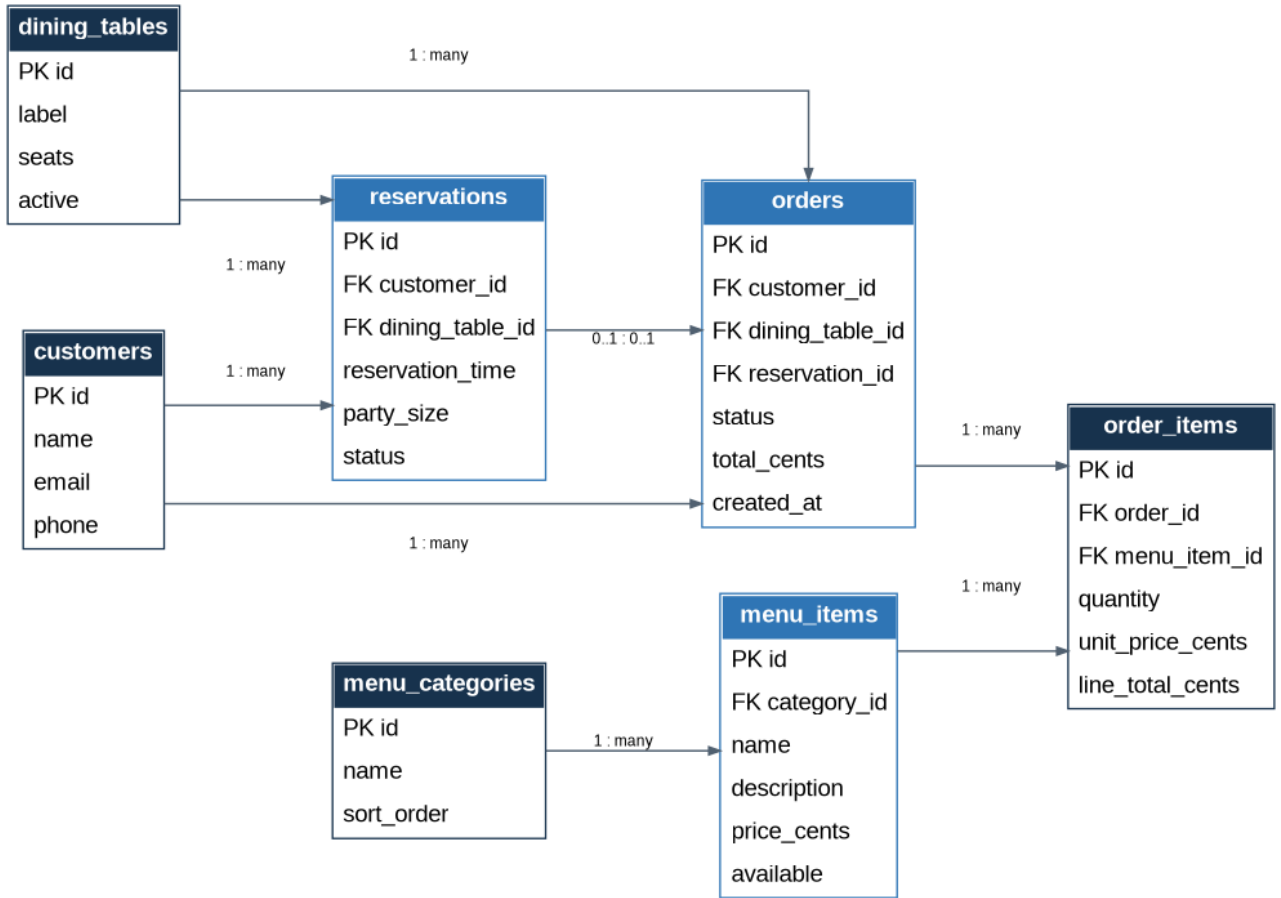
THE OPENAPI SPEC DEFINES WHAT THE INTERFACE MUST DO. THE CODE MUST IMPLEMENT IT.

2. STANDARD TECHNOLOGY — DO NOT CHOOSE A NEW STACK

- Python 3.
- Flask for the HTTP service.
- Python built-in sqlite3 for the database.
- OpenAPI YAML as the source of truth.
- openapi-core (or the company-provided equivalent) for request/response contract validation.
- pytest + a normal HTTP test client for tests.
- Company Git repository; deploy through the existing AWS path only if already provided.
- Maximum practical use of AI is expected for scaffolding, debugging, test generation and review. Reasonable AI/API spend is reimbursed.

3. CANONICAL RESTAURANT DATA MODEL

Use exactly this simple Restaurant model. Do not add inventory, employees, kitchen screens, delivery, loyalty, tax engines or payment processing.



Canonical Restaurant ER model for this SOW

4. TABLE CONTRACT

TABLE	FIELDS
customers	id PK; name; email UNIQUE; phone
dining_tables	id PK; label UNIQUE; seats; active
reservations	id PK; customer_id FK; dining_table_id FK; reservation_time; party_size; status
menu_categories	id PK; name UNIQUE; sort_order
menu_items	id PK; category_id FK; name; description; price_cents; available
orders	id PK; customer_id FK nullable; dining_table_id FK nullable; reservation_id FK nullable; status; total_cents; created_at
order_items	id PK; order_id FK; menu_item_id FK; quantity; unit_price_cents; line_total_cents

5. REQUIRED OPENAPI OPERATIONS

operationId	HTTP	PURPOSE
listMenu	GET /menu	Return available menu items grouped/identified by category.
getMenuItem	GET /menu/{id}	Return one menu item or 404.
createCustomer	POST /customers	Create a customer; reject duplicate email.
listDiningTables	GET /tables	Return active dining tables and seats.
createReservation	POST /reservations	Reserve a table for a customer/date/time/party size.
getReservation	GET /reservations/{id}	Return reservation details.
createOrder	POST /orders	Create order + order_items transactionally; server owns prices/totals.
getOrder	GET /orders/{id}	Return complete order including item details.
updateOrderStatus	PATCH /orders/{id}/status	Apply allowed status transition.
listCustomerOrders	GET /customers/{id}/orders	Return order history for one customer.

6. RESTAURANT BUSINESS RULES

- Menu price comes from SQLite. createOrder must never trust a price supplied by the client.
- Order total = sum(quantity × database menu price) and is calculated by the backend.
- Unavailable menu items cannot be ordered.
- Reservation party_size must be > 0 and cannot exceed the selected table seats.
- The same dining table cannot have two active reservations for the same reservation_time in this simple model.
- Order status is one of NEW, PREPARING, READY, COMPLETED, CANCELLED.
- Allowed transitions: NEW→PREPARING or CANCELLED; PREPARING→READY; READY→COMPLETED. Other transitions return a contract-defined error.
- Historical order_items preserve unit_price_cents even if menu price changes later.

7. WHAT THE HARNESS MUST DO

THE HARNESS IS REUSABLE. IT MUST CONTAIN ZERO RESTAURANT-SPECIFIC BUSINESS LOGIC.

- Validate that openapi.yaml is syntactically valid before the application starts.
- Create/reset a clean SQLite database from schema.sql and seed.sql for each test run.
- Start the HTTP application in a predictable local test mode.
- Run tests against the HTTP interface, not directly against handler functions.
- Validate requests and responses against openapi.yaml, including status codes and response schemas.
- Provide a simple command such as `make test` or `./run-tests.sh` that a Fresher can run repeatedly.
- Print clear PASS/FAIL output showing the failing operationId and useful validation/test error.
- Keep test database files disposable; no shared external database or credentials.
- Allow the same Harness to be reused later by replacing the Restaurant openapi.yaml/schema/tests with another backend model.

8. REPOSITORY LAYOUT

```
restaurant-api/
  openapi.yaml           # supplied/approved contract
  schema.sql
  seed.sql
  src/
    app.py
    db.py
    handlers.py
  harness/
    validate_openapi.py
    reset_db.py
    contract.py
  tests/
    test_menu.py
    test_reservations.py
    test_orders.py
  requirements.txt
  run-tests.sh (or Makefile)
  README.md
```

9. HARNESS/IMPLEMENTATION BOUNDARY

HARNESS — REUSABLE

- OpenAPI validation
- SQLite reset/init helper
- HTTP contract validation

RESTAURANT IMPLEMENTATION — REPLACEABLE

- Restaurant schema/seed data
- Restaurant handlers/SQL
- Restaurant business rules

- test runner + PASS/FAIL reporting
- common fixtures/helpers

- Restaurant endpoint tests
- Restaurant OpenAPI contract

10. HOW TO BUILD IT — AI-FIRST SIX-HOUR PLAN

TIME	PHASE	HOW TO / USE AI	EXIT GATE
0:00–0:30	Read Contract	Give AI openapi.yaml, ER model and SOW. Have it enumerate operations, schemas, SQLite tables, foreign keys and business rules. Do not code until every operationId is understood.	Endpoint checklist complete.
0:30–1:15	DB + Seed	Use AI to generate/review schema.sql and seed.sql exactly from the SOW. Create tables, foreign keys, indexes/constraints and small seed menu/table/customer data. Verify reset creates a clean DB.	SQLite reset PASS.
1:15–2:00	Build Harness	Implement OpenAPI validation, DB reset, application launch/test client, request/response validation and one-command test runner. Prove it with one intentionally valid and one intentionally invalid response.	Harness detects mismatch.
2:00–3:15	Implement APIs	Use AI aggressively to scaffold Flask routes/handlers and parameterized SQLite SQL. Implement the 10 operations. Keep SQL/business logic readable; do not introduce ORM/framework complexity.	All endpoints callable.
3:15–4:15	Contract Tests	Use AI to generate tests from the OpenAPI examples and business rules. Test normal path plus at least: duplicate email, missing menu item, oversized reservation, duplicate table/time reservation, unavailable item, invalid order transition.	Contract/business tests run.
4:15–5:00	Triage + Fix	Run full harness. Feed exact failures to AI one at a time. Fix implementation or spec mismatch without weakening tests. Re-run until clean.	Full suite PASS.
5:00–5:30	Dogfood	A second Fresher follows README only: create customer → list menu → reserve table → create order → advance status → read order history. They must not inspect source first.	Second user succeeds.
5:30–6:00	Review + Deliver	Give AI Git diff + test output; ask for only correctness/security-of-SQL/basic-quality issues, no new features. Run from clean DB, commit/push, deploy through existing AWS path if provided.	Git + final PASS.

11. REQUIRED ACCEPTANCE TESTS

ID	TEST	PASS CONDITION
A1	OpenAPI Valid	openapi.yaml validates before test execution.
A2	Clean DB	Harness recreates SQLite deterministically from schema.sql + seed.sql.
A3	Contract Judge	An intentionally wrong response body/status is caught by the Harness.
A4	Menu	Menu endpoints return only contract-valid menu data.
A5	Customer	Customer creation works; duplicate email is rejected as specified.
A6	Reservation	Valid reservation succeeds; party-too-large and duplicate table/time are rejected.
A7	Order Transaction	Order + items are written together; prices/totals come from database, not caller.
A8	Order Read	getOrder returns joined order + items matching OpenAPI response schema.
A9	State Machine	Allowed status transitions succeed; invalid transition fails predictably.
A10	History	listCustomerOrders returns the customer's created order.
A11	Fresh DB Repeat	Run full suite twice from fresh SQLite; both runs PASS.
A12	Reusability	Harness files contain no Restaurant table/menu/order-specific business logic.
A13	Dogfood	Second Fresher completes the canonical workflow using README + HTTP interface.

12. CANONICAL FRESHER USER TEST

1. Start from a clean database.
2. GET /menu.
3. POST /customers – create Alice.
4. GET /tables – choose a 4-seat table.
5. POST /reservations – Alice, party of 2.
6. POST /orders – Alice orders 2 menu items.
7. GET /orders/{id} – verify server total.
8. PATCH status NEW → PREPARING → READY → COMPLETED.
9. GET /customers/{id}/orders – verify completed order.
10. Run the entire automated Harness – PASS.

The second Fresher must be able to perform this from the README and OpenAPI contract without being coached through the source code.

13. DELIVERABLES BY HOUR 6

- Working reusable OpenAPI + SQLite development Harness.
- Canonical Restaurant ER model, schema.sql and seed.sql.
- Approved openapi.yaml implementing the 10 required operations.

- Working Python/Flask/SQLite backend that conforms to the OpenAPI contract.
- Automated contract + business-rule tests with clear PASS/FAIL output.
- One-command clean test run.
- Second-Fresher dogfood test completed.
- Code/config committed and pushed to company Git.
- Deployment through the existing AWS path if that path is supplied; do not build new infrastructure.
- Short README: setup, reset DB, run service, run tests, inspect API, add a new endpoint.

14. OUT OF SCOPE

- Frontend or mobile UI.
- Authentication/security platform work.
- Inventory/ingredients/recipes.
- Employees/waiters/kitchen display.
- Payments, taxes, tips or POS integration.
- Delivery/takeout logistics.
- ORMs, microservices, Redis, Kafka, NATS or external databases.
- MCP server/gateway implementation. Good operationId/descriptions keep the OpenAPI contract ready for later MCP work.
- Autonomous GitHub AI iteration/orchestration — that belongs to Product 003, not this Fresher assignment.

15. DEFINITION OF DONE

```

OPENAPI CONTRACT
  ↓
FRESHER IMPLEMENTS BACKEND
  ↓
SQLITE
  ↓
HARNESS: SPEC + DB RESET + HTTP + CONTRACT TESTS + BUSINESS TESTS
  ↓
PASS

PROOF: SECOND FRESHER CAN USE THE RESTAURANT API FROM README + OPENAPI.

```

IF THE HARNESS ONLY WORKS FOR THE RESTAURANT, IT IS NOT A HARNESS.

16. SIX-HOUR RULE

The six hours include understanding OpenAPI, schema/seed, Harness implementation, all Restaurant endpoints, contract/business testing, AI-assisted triage/fixing, second-Fresher user test, Git delivery and existing-path deployment if provided.

NO SEVENTH HOUR. NO “ALMOST DONE.” CONTRACT PASS OR PRODUCT FAILED.